

Problems and fixes for the current Implementation:

1: f_process_trend Is Not Measuring Process Trend

Current Code

File:

analysis/extractor_engine.py

Current implementation:

```
process_history[pid].append(cpu)
```

```
process_trend = 0
```

```
if len(process_history[pid]) >= 2:
```

```
    process_trend = (  
        process_history[pid][-1]  
        -  
        process_history[pid][-2]  
    )
```

Returned as:

```
"f_process_trend": process_trend
```

Why This Is Wrong

The feature name suggests:

```
f_process_trend
```

but the implementation actually computes:

CPU Trend

Rabbit Worms are defined by:

Process Replication not CPU utilization.

A Rabbit Worm can create thousands of sleeping processes while maintaining relatively low CPU usage.

In that situation:

CPU Trend $\approx$ 0

while:

Process Growth $\rightarrow$ Exploding

The current implementation would underestimate the threat.

Correct Rabbit-Oriented Implementation

Maintain history of process tree size.

Example:

```
tree_history = defaultdict(  
    lambda: deque(maxlen=20)  
)  
  
tree_history[pid].append(  
    process_tree_size  
)  
  
process_trend = 0  
  
if len(tree_history[pid]) >= 2:  
    previous = tree_history[pid][-2]  
  
    if previous > 0:  
        process_trend = (  
            process_tree_size/previous  
        )
```

Returned:

```
"f_process_trend": process_trend
```

We can keep CPU trend.

Rename it:

f_cpu_trend

Then add:

f_process_trend

for process/entity growth.

That makes the theory and implementation consistent.

Why This Fix Is Correct

This directly measures:

Entity Growth

which is the defining Rabbit Worm characteristic.

2: f_proc_spawn

Current Code

File:

analysis/extractor_engine.py

Current implementation:

```
spawn_history[pid].append(
```

```
    process_tree_size
```

```
)
```

```
process_growth = (
```

```
    current_tree_size
```

```
-
```

```
    previous_tree_size
```

```
)
```

```
"f_proc_spawn": process_growth
```

Why This Is Wrong

This computes:

Tree Size Delta

not:

Spawn Rate

The feature name implies:

Processes Created Per Interval

but the code measures:

Change in Entity Size

Correct Rabbit-Oriented Implementation

Track actual child creation count.

Example:

```
new_children_created = (  
    current_tree_size  
    -  
    previous_tree_size  
)  
  
spawn_rate = (  
    new_children_created / MONITOR_INTERVAL( delta t)  
)
```

Return:

```
"f_proc_spawn": spawn_rate
```

Why This Fix Is Correct

Rabbit Worms should be detected from:

Processes Created / Second

rather than raw tree size growth.

3: Trust Vector Ignores Rabbit Features

Current Rabbit Features:

f_proc_spawn
f_proc_tree
f_process_trend

influence classification but do not directly contribute to trust evolution.

Current Code

File:

analysis/trust_engine.py

Current Trust Dimensions:

cpu
memory
threads
connections
files

Dynamic Trust:

dynamic_trust =

```
average(  
    cpu,  
    memory,  
    threads,  
    connections,  
    files  
)
```

Why This Is Wrong

Rabbit-specific features:

f_proc_spawn

f_proc_tree

f_process_trend

are completely absent from trust computation.

They only influence:

worm_classifier.py later.

Consequence

Trust can remain relatively healthy while:

Process replication is exploding

This weakens the entire trust model.

Correct Design

Add Rabbit Trust Components:

spawn_trust

tree_trust

trend_trust

Updated Trust Vector:

[

Tcpu,

Tmem,

Tthread,

Tconn,

Tfile,

Tspawn,

Ttree,

Ttrend

]

Why This Fix Is Correct

Rabbit Worm behaviour should directly degrade trust.

Currently trust reflects:

Resource Impact

but not:

Replication Behaviour

4: EMA Is Too Conservative

Current Code

File:

analysis/trust_engine.py

Current:

EMA_ALPHA = 0.9

Meaning:

90% previous trust

10% new evidence

Problem

Rabbit Worms grow exponentially.

Trust drops too slowly.

Recommended Fix

EMA_ALPHA = 0.7

We can check for 2-3 values and see how our model performs.

This matches the PPT and allows faster adaptation.

5: Persistence Delays Rabbit Detection

Current Code

File:

analysis/persistence_engine.py

Current:

```
self.delta = 3
```

Three suspicious observations required.

Problem

Rabbit Worms double rapidly.

Waiting:

3 monitoring cycles can allow thousands of new processes.

Recommended Fix

Introduce Rabbit bypass:

```
if (  
    spawn_score > 0.8  
    and  
    tree_score > 0.8  
    and  
    trend_score > 0.8  
):  
    return {  
        "persistent": True,  
        "stage": "terminate"  
    }
```

This bypasses persistence only for strong Rabbit signatures.

6: Self-Healing Is Disabled

Current implementation is running in evaluation mode.

SAFE_MODE prevents active containment actions from being executed.

This is appropriate during development but prevents evaluation of the complete self-healing pipeline.

Current Code

File:

analysis/response_engine.py

Current:

```
SAFE_MODE = True
```

Followed by:

```
if SAFE_MODE:
```

```
    return {  
        ...  
        "status":  
            "safe mode (healing disabled)"  
    }
```

Consequence

The framework performs:

Detection

Classification

Trust Update

Logging

but never performs:

Restriction

Isolation

Termination

Recommendation

Keep:

SAFE_MODE = True

during development.

For evaluation experiments:

SAFE_MODE = False

inside an isolated VM.

7: Syscall Frequency Is Not Real Syscall Frequency

Current Code

File:

analysis/extractor_engine.py

Current:

```
syscall_proxy = (
```

```
    cpu
```

```
    +
```

```
    threads
```

```
    +
```

```
    connection_count
```

```
    +
```

```
    file_events
```

```
)
```

Returned as:

```
"f_syscall_freq"
```

Problem

This is not syscall frequency.

It is a behavioural proxy.

Recommendation

Either:

Option A

Rename:

"f_behavior_score"

or

Option B

Collect actual:

fork()

clone()

vfork()

events.

For Rabbit Worm research, Option B is preferred.